SGX Key Recovery Setup Guide

Satwik Raj Wadhwa

April 28, 2026

1 Clone the Repository

```
git clone https://github.com/Satwik-Raj/sgx-step.git
cd sgx-step
```

2 System Requirements

Linux kernel parameter	Motivation
`nox2apic`	Optionally configure local APIC device in memory-mapped I/O mode (to make use of SGX-Step's precise single-stepping features). If set, make sure to set X2APIC to 0 in libsgxstep/config.h. Alternatively, you can leave the CPU in x2APIC mode and recent SGX-Step distributions should also be compatible.
iomem=relaxed no_timer_check	Suppress unneeded warning messages in the kernel logs.
nmi_watchdog=0	Suppress the kernel NMI watchdog.
isolcpus=1	Affinitize the victim process to an isolated CPU core.
clearcpuid=308,295,514	Disable supervisor mode access prevention (SMAP, bits 295), supervisor mode execution prevention (SMEP, bits 308) and user-mode instruction prevention (UMIP, bits 514) features.
pti=off	Disable Kernel Page-Table Isolation (to avoid kernel panics with user IRQ handlers).
rcupdate.rcu_cpu_stall_suppress=1	Disable the kernel's read-copy update (RCU) CPU stall detector (to avoid warnings when single-stepping for a long time without calling the kernel's timer interrupt handler.)
msr.allow_writes=on	Suppress kernel warning messages for model-specific register (MSR) writes by SGX-Step.
vdso=0	Only on recent Linux kernels: disable vdso_sgx_enter_enclave library (not compatible with AEP interception patches).
dis_ucode_ldr	Optionally disable CPU microcode updates (recent transient-execution attack mitigations may necessitate re-calibrating the single-stepping interval).

Table 1: Linux kernel parameters and their motivations

Pass the desired boot parameters to the kernel as follows:

```
$ sudo vim /etc/default/grub
  # Add the following line: GRUB_CMDLINE_LINUX_DEFAULT="quiet splash iomem=relaxed
      no_timer_check clearcpuid=308,295,514 pti=off isolcpus=1 nmi_watchdog=0 rcupdate
      .rcu_cpu_stall_suppress=1 msr.allow_writes=on vdso=0"
$ sudo update-grub && reboot
```

3 Build and load /dev/sgx-step

SGX-Step comes with a loadable kernel module that exports an IOCTL interface to the libsgxstep user-space library. The driver is mainly responsible for (i) hooking the APIC timer interrupt handler, (ii) collecting untrusted page table mappings, and optionally (iii) fetching the interrupted instruction pointer for benchmark enclaves.

To build and load the /dev/sgx-step driver, execute:

```
$ cd kernel/
$ ./install_SGX_driver.sh
$ make clean load
```

In case you reboot anytime, just run this to get it back up:

```
$ cd kernel/
$ make clean load
```

4 Patch and install SGX SDK

To enable easy registration of a custom Asynchronous Exit Pointer (AEP) stub, SGX-Step modified the untrusted runtime of the official Intel SGX SDK.

```
$ cd sdk/intel-sdk/
$ ./install_SGX_SDK.sh
$ echo 'source /opt/intel/sgxsdk/environment' >> ~/.bashrc
$ source ~/.bashrc
$ sudo service aesmd status # stop/start aesmd service if needed
```

5 Checking if system is ready

To check that the currently running kernel is configured correctly, execute:

```
$ sudo ./check_sys.sh
.. Checking recommended SGX-Step parameters [OK]
.. Checking unknown kernel parameters [OK]
.. Checking CPU features [OK]
.. Checking kernel page-table isolation [OK]
```

6 Test Single Stepping

To test timer single-stepping functionality, try for example building and running a benchmark enclave to single-step a slide of 100 successive nop instructions:

```
$ cd app/bench
$ NUM=100 make parse
```

```
-----------------------------------------------------------------
[main.c] all done; counted 187/188 IRQs (AEP/IDT)
-----------------------------------------------------------------

parse_nop.py: found instruction slide at 0x6000 (length=100)
parse_nop.py counted tot= 108  one= 100  zero= 7  plus= 0
```

Here, tot = total irqs, one = single stepping, zero = zero stepping, plus = multistepping

Configure the timer value in "libsgxstep/config.h" : decrease if you get multistepping, increase if you get stuck in an infinite loop because the timer value is shorter than the irq handling time.

```
#define SGX_STEP_TIMER_INTERVAL 32
```

7 Insert the Module for accessing Access bit from PTE

```
$ cd check_access_bit/
$ sudo ./install_and_check_module.sh
[*] Checking if module is already loaded...
[*] Module already loaded. Skipping insmod.
[*] Sleeping for 1 second...
[*] Compiling check_logic.c...
[*] Running check_logic...
[*] Output from check_logic:
After clear:
Accessed bit = 0
0
After access:
Accessed bit = 1
```

8 Running SGX KEY RECOVERY Project

Run the following to recover the key from outside the enclave 100 times, and print avg accuracy

```
$ cd app/sgx-key-recovery/
$ sudo ./check_accuracy.sh
```

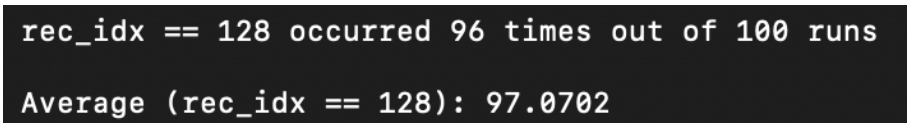

Figure 1: Output over 100 runs Screenshot

Run the following to recover the bits one time manually

```
$ cd app/sgx-key-recovery/
$ make clean
$ make
$ sudo ./app
```

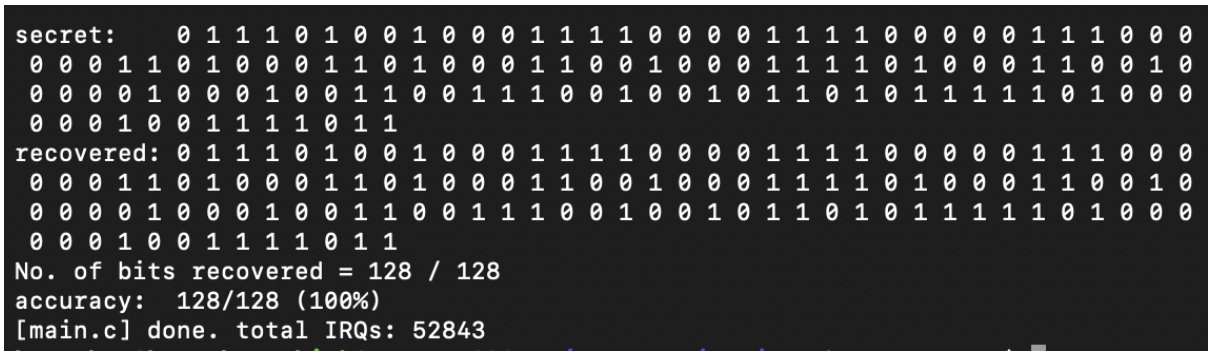

Figure 2: Single Run Output Screenshot

9 Project Structure

```
sgx-step/
|--check_sys.sh

|--kernel/
| |--install_SGX_driver.sh

|--sdk/
| |--intel-sdk/
| | |--install_SGX_SDK.sh

|--libsgxstep/
| |--config.h
| |--enclave.h
| |--enclave.c

|--check_access_bit/
| |--install_and_check_module.sh

|--app/
| |--bench/
| | |--benchmark-related-sources

| |--sgx-key-recovery/
| | |--Makefile
| | |--main.c
| | |--check_accuracy.sh
| | |--Enclave/
| | | |--encl.edl
| | | |--encl.c
```

10 Notes

- To run the enclave in debug mode, convert the second parameter from 0 to 1 in line 187 "app/sgx-key-recovery/main.c"

```
SGX_ASSERT(sgx_create_enclave("./Enclave/encl.so",0,&token,&updated,&eid,NULL));
```

- To add your custom aep_handler in place the current one, replace `aep_cb_func` in line 198 "app/sgx-key-recovery/main.c"

```
    register_aep_cb(aep_cb_func);
```

- This command starts the interrupt timer.

```
    apic_timer_irq(SGX_STEP_TIMER_INTERVAL);
```

- The EDBGRD(Enclave DeBuG ReaD) instruction helps in reading the SSA registers of the enclave from outside the enlcave given it is running in debug mode.

- The following lines are from "libsgxstep/enclave.h" which declare the corresponding function which we can use to directly access enclave rip from aep_cb_func. Corresponding definitions can be found in "libsgxstep/enclave.c"

```
uint64_t edbgrd_ssa_gprsgx(int gprsgx_field_offset);
#define edbgrd_erip() edbgrd_ssa_gprsgx(SGX_GPRSGX_RIP_OFFSET)
```

- To add any ECALL or OCALL for the enclave, add their declarations in "app/sgx-key-recovery/Enclave/encl.edl"
- We can't call any external (untrusted) function from inside the enclave unless it is mentioned in "encl.edl" file.
- Any interaction with the outside must be explicitly defined in the EDL (Enclave Definition Language) file, such as encl.edl.